Lab 04 — HTTP vs HTTPS: Plaintext vs Encrypted Traffic

Protocols: HTTP (HyperText Transfer Protocol) + TLS (Transport Layer Security)
Tool: Wireshark on Kali Linux + Apache2 on Ubuntu
RFC: HTTP/1.1 — RFC 7230 | TLS 1.3 — RFC 8446
Difficulty: Intermediate
Time Taken: ~2-3 hours

Objective

Capture HTTP traffic to a local Apache server and contrast it against HTTPS/TLS traffic. This lab demonstrates the most impactful security concept for web developers and penetration testers: **HTTP exposes everything — credentials, cookies, and data — in plaintext. HTTPS encrypts it all.**

Protocol Background

HTTP — HyperText Transfer Protocol

HTTP is the foundation of the World Wide Web. It's a **request-response protocol** at Layer 7 (Application Layer), operating over TCP port 80.

HTTP Request Structure:

```
GET /login HTTP/1.1

Host: 192.168.x.x

User-Agent: Mozilla/5.0 (...)

Accept: text/html

Cookie: session=abc123; auth=xyz789

Connection: keep-alive


[Optional Body — e.g., POST data]

username=admin&password=secret123
```

HTTP Response Structure:

```
HTTP/1.1 200 OK

Content-Type: text/html

Set-Cookie: session=newtoken; HttpOnly

Content-Length: 1024


<!DOCTYPE html>
```

<html>...page content...</html>

The critical problem: Every word of both the request and response is sent in **raw plaintext** over the network. Anyone with Wireshark on the same network sees all of it.

HTTPS — HTTP over TLS

HTTPS wraps HTTP inside **TLS (Transport Layer Security)**, encrypting the entire application layer. The TCP handshake is visible, but all HTTP content (headers, URLs, cookies, passwords, response bodies) is encrypted.

HTTP: [IP Header][TCP Header][HTTP Headers + Body] ← ALL VISIBLE

HTTPS: [IP Header][TCP Header][TLS Record (ENCRYPTED)] ← ONLY METADATA VISIBLE

What's still visible even in HTTPS:

- Source and destination IP addresses
- Destination port (443)
- **SNI (Server Name Indication)** — the hostname in the TLS ClientHello
- Packet sizes and timing (traffic analysis)
- Certificate information (server's domain, issuer, validity)

Lab Setup

Apache2 HTTP Server on Ubuntu

```
# Install Apache2 on Ubuntu VM
sudo apt install apache2 -y
sudo systemctl start apache2

# Create a test login page
sudo nano /var/www/html/login.html
# Add a simple HTML form with username/password fields

# Verify it's running
curl http://localhost/login.html
```

Capturing from Kali

```
# On Kali, browse to Ubuntu's Apache server
curl http://192.168.x.x/login.html
```

```
# POST credentials (simulates login form submission)
curl -X POST http://192.168.x.x/login.html \
  -d "username=admin&password=password123"
```

Wireshark running on Kali captures everything.

Wireshark Filters Used

```
# All HTTP traffic
http

# HTTP GET requests only
http.request.method == "GET"

# HTTP POST requests (form submissions, logins)
http.request.method == "POST"

# HTTP responses
http.response

# Search for credentials in HTTP POST data
http.request.method == "POST" && tcp.port == 80

# TLS/HTTPS traffic
tls

# TLS ClientHello (beginning of handshake)
tls.handshake.type == 1

# TLS ServerHello
tls.handshake.type == 2

# TLS Application Data (encrypted payload)
```

tls.record.content_type == 23

Combined HTTP and HTTPS

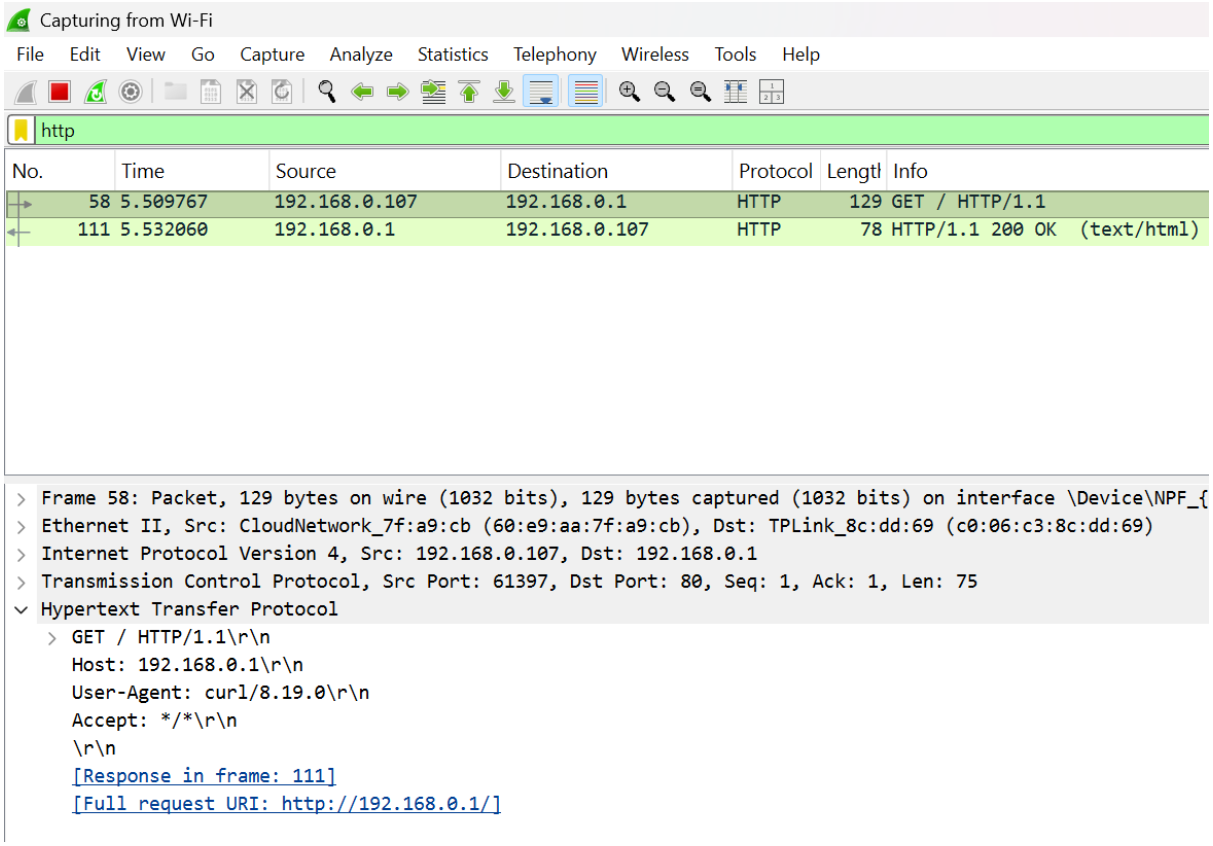
http || tls

Follow entire HTTP stream

Right-click packet → Follow → HTTP Stream

Screenshots & Findings

Screenshot 1 — HTTP Packet Capture (Full Plaintext Visible)



What this shows:

- Wireshark fully decodes the HTTP request/response
- **All headers visible in plaintext:** Host, User-Agent, Cookie, Content-Type
- **URL fully visible:** GET /login.html HTTP/1.1
- **If POST request:** Form data (username, password) visible in packet body

Follow HTTP Stream (Right-click → Follow → HTTP Stream) shows the entire conversation:

→ GET /index.html HTTP/1.1

Host: 192.168.x.x

User-Agent: curl/7.x

Accept: */*

← HTTP/1.1 200 OK

Date: [timestamp]

Server: Apache/2.4.x (Ubuntu)

Content-Length: 1024

Content-Type: text/html

<!DOCTYPE html>...

Critical — What an attacker captures in a POST login request:

POST /login HTTP/1.1

Host: 192.168.x.x

Content-Type: application/x-www-form-urlencoded

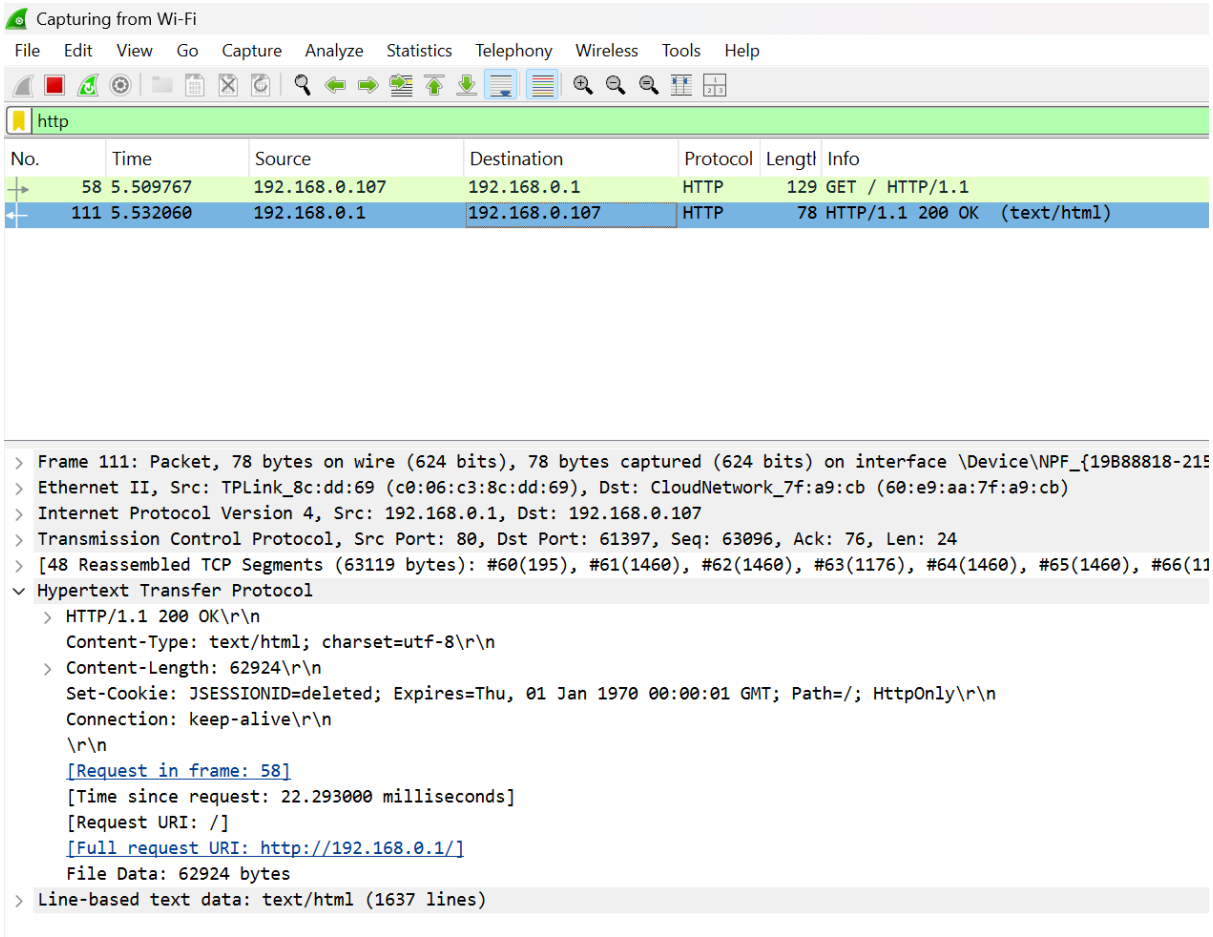
Content-Length: 35

Cookie: PHPSESSID=abc123

username=admin&password=Password123!

The password is completely visible in Wireshark. No decryption needed.

Screenshot 2 — TLS 1.3 Handshake (HTTPS)



What this shows:

- TLS ClientHello packet from browser to server
- **SNI field visible:** server_name: google.com (or your server's hostname)
- Cipher suites offered by the client
- **After handshake completes:** All subsequent packets show as "Application Data" — unreadable

TLS 1.3 Handshake Sequence visible in Wireshark:

1. Client → Server: ClientHello

  - TLS version

  - Supported cipher suites

  - Random bytes (for key derivation)

  - SNI (server name) ← STILL VISIBLE IN PLAINTEXT

2. Server → Client: ServerHello + Certificate + Finished

- Chosen cipher suite
- Server's TLS certificate (domain, issuer, public key)
- Encrypted extensions

3. Client → Server: Finished (encrypted)

4. [All further packets]: Application Data (encrypted)
   - Content looks like random bytes
   - Cannot be read without the session keys

Key observation: Even though HTTPS encrypts the content, the **SNI field in ClientHello** still reveals which domain you're connecting to. ESNI/ECH (Encrypted Client Hello) is the next evolution to hide this too.

HTTP vs HTTPS Comparison — What Wireshark Shows

Information	HTTP (Port 80)	HTTPS (Port 443)
URL path	✅ Fully visible	❌ Encrypted
Request headers	✅ Fully visible	❌ Encrypted
Cookies & session tokens	✅ Fully visible	❌ Encrypted
POST data (passwords)	✅ Fully visible	❌ Encrypted
Response body (page HTML)	✅ Fully visible	❌ Encrypted
Destination IP	✅ Visible	✅ Visible
Destination hostname	✅ In Host header	⚠️ In SNI field
Server software	✅ In Server header	❌ Encrypted
TLS certificate	N/A	✅ Visible (metadata only)
Packet sizes/timing	✅ Visible	✅ Visible

Security Implications of HTTP

1. Credential Theft — Live Demo

If any website (even internal tools) uses HTTP for login:

1. Attacker runs Wireshark (or tcpdump -A port 80) on the local network
2. User submits login form
3. Attacker sees the POST request with username and password in plaintext
4. **No hacking required — just passive observation**

Attacker's tcpdump command to grab HTTP POST data

tcpdump -i eth0 -A 'tcp port 80' | grep -i "password\|user\|login\|pass"

2. Session Hijacking (Cookie Theft)

Even after a user logs in over HTTPS, if the session cookie is **served without the Secure flag**, it can be transmitted over HTTP and captured:

Captured Cookie: session=eyJhbGciOiJIUzI1NiJ9.xxxxx

Attacker replays this cookie → **authenticated as the victim without knowing their password.**

3. HTTP Response Manipulation (SSL Stripping)

Tool: **sslstrip** (by Moxie Marlinspike)

Attack flow:

1. Attacker performs ARP spoofing (Lab 5) to become MITM
2. sslstrip intercepts HTTPS redirects and downgrades them to HTTP
3. Victim communicates over HTTP thinking they're on HTTPS
4. Attacker sees all traffic in plaintext

Why this matters: The browser's padlock icon disappears, but many users don't notice.

4. HTTP Header Information Disclosure

Even a simple GET request response leaks:

Server: Apache/2.4.41 (Ubuntu)

X-Powered-By: PHP/7.4.3

This tells an attacker exactly which software versions to look up CVEs for.

Defender Perspective

Enforcing HTTPS — Key Headers

HTTP Strict Transport Security (HSTS)

Forces browser to ONLY use HTTPS for this domain (max 1 year)

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

```
# Secure cookie flags
Set-Cookie: session=token; Secure; HttpOnly; SameSite=Strict
# Secure: only sent over HTTPS
# HttpOnly: not accessible by JavaScript (prevents XSS theft)
# SameSite=Strict: not sent in cross-site requests
```

Apache Configuration for HTTPS

```
# Redirect all HTTP to HTTPS
<VirtualHost *:80>
   ServerName yourdomain.com
   Redirect permanent / https://yourdomain.com/
</VirtualHost>

# Disable old TLS versions
SSLProtocol -all +TLSv1.2 +TLSv1.3
SSLCipherSuite HIGH:!aNULL:!MD5
```

Detecting SSL Stripping in Wireshark

```
# Look for HTTP requests where referer was HTTPS
http.referer contains "https://"

# HTTPS downgrade: expect HTTPS, getting HTTP
http && ip.dst == [trusted server IP]
```

Key Takeaways

1. **HTTP is forensically transparent** — every password, cookie, and page content is readable in Wireshark
2. **HTTPS encrypts the content but not the metadata** — destination IPs and SNI still leak
3. **Session cookies without the Secure flag** can be stolen over HTTP even if login was HTTPS
4. **HSTS** is the browser-level defense against SSL stripping
5. **TLS 1.3** is significantly more secure than 1.2 — shorter handshake, forward secrecy by default
6. **Any internal tool on HTTP** (even unreachable from internet) is vulnerable to insider threats

---